94.3
Pass

Vinit Virendra Bolla

PDF generated at: 24 Feb 2026 10:16:46 UTC

View this report on HackerRank

Score

94.3% • 165.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 46 min 25 sec on 24 Feb 2026 14:57:50 IST

Candidate Information

Email	vinitbolla10@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	24 Feb 2026 14:57:50 IST
Time taken	46 min 25 sec/ 90 min
Work Experience	5 years
Invited by	Vipul
Invited on	20 Feb 2026 13:12:59 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	0%

2	Spring Boot Intermediate	100%	
3	Problem Solving Intermediate	100%	
4	Java Intermediate	100%	

Tags Distribution

Spring Boot	60%	Hard	0%
TDD	0%	Spring	100%
Application Properties Annotations	100%	Medium	100%
Exception Handling	100%	Aspect-Oriented Programming	100%
Sorting	100%	Greedy Algorithms	100%
Two Pointers	100%	Greedy	100%
Java	100%		

Questions

Spring Boot (Advanced) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status

1	Determine the Output Multiple Choice	Spring Boot (Advanced)	0/5	-	3 min 2 sec	⊗
2	Response Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 39 sec	⊗
3	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 49 sec	✓
4	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 27 sec	✓
5	Predict the Output Multiple Choice	Spring Boot (Intermediate)	5/5	-	5 min 4 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Maximum Array Correlation Coding	Problem Solving (Intermediate)	75/75	-	24 min 9 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: Optimized Sequential Video Watching Coding	Java (Intermediate)	75/75	-	8 min 43 sec	

1. Determine the Output

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringQuestionsApplication.class, args);
    }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is behind the lies");
        System.out.println("This is working perfectly");
        throw new Exception("Does this work?");
    }
}
```

Truth.java

```
package com.hackerrank.spring.reality;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is in your eyes");
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

Interviewer guidelines

Correct Answer:

Option [A] – "Truth is in your eyes"

Explanation: Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

Incorrect Answers:

Option [B] – Multiple prints + exceptions

Explanation: `Lie` never runs, so its prints and exception do not occur.

Option [C] – Prints from Lie only

Explanation: `Truth` is discovered and executed; `Lie` is not.

Option [D] – Prints from Lie, then Truth

Explanation: `Lie` is outside the scan path and never executed.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Truth is in your eyes ✓

- Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops
- **Truth is behind the lies This is working perfectly And exception occurs and application stops**
- Truth is behind the lies This is working perfectly And exception occurs containing “Does this work?” Truth is in your eyes

No comments.

2. Response

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

Which response will cause the following test case to fail?

```
@Test
   @DisplayName("Filter Price Check 3")
   public void FilterPriceCheck3() throws Exception {
      mockMvc.perform(get("/filter/price/150/500"))
            .andDo(print())
            .andExpect(jsonPath("$[*].product", containsInAnyOrder("Product 1", "Product 2" , "Produc
t 5")))
   }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] [{"product" : "Product 5"} , {"product" : "Product 1"}, {"product" : "Product 2"}]
- [ ] [{"product" : "Product 5"} , {"product" : "Product 2"}, {"product" : "Product 1"}]
- [x] **[{"product" : "Product 1"} , {"product" : "Product 2"}, {"product" : "Product 5"}]**
- [ ] [{"product" : "product 5"} , {"product" : "product 2"}, {"product" : "product 1"}]

No comments.

---

3. Configuration Properties

Correct

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
```

```
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).

- [ ] The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).

- [x] ** A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final. **

- [ ] An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.

- [ ] A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

4. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }
- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }** ✓
- None of the above

No comments.

5. Predict the Output

Correct

Multiple Choice | Medium | Exception Handling | Spring Boot | Aspect-Oriented Programming

Question description

Consider the following Snippet and predict the output.

The following API is triggered: http://localhost:8080/add/employee?eId=123&fName=Ravi&lastName=Shekhar

```
@Autowired
private EService eService;

@RequestMapping(value = "/add/employee", method = RequestMethod.GET)
public com.model.Employee add(@RequestParam("eId") String eId, @RequestParam("fName") Strin
g fName, @RequestParam("lastName") String lastName)
{
   return eService.createEmployee(eId, fName, lastName);
}

@Aspect
@Component
public class EServiceAspect
{
   @After(value = "execution(* com.random.service.EService.*(..)) && args(eId, fName, lastName)")
   public void afterAdvice(JoinPoint joinPoint, String eId, String fName, String lastName) {
      System.out.println("After method:" + joinPoint.getSignature());
      System.out.println("Creating Employee with first name - " + fName+ ", second name - " + lastNa
me+ " and id - " + eId);
   }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] After method : com.test.model.Employee addEmployee(String,String,String)
Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(string,string,string)
Creating Employee with first name - Ravi , second name - Shekhar and id - 123

After method : com.test.model.Employee addEmployee(empId,firstName,secondName) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Shekhar, second name - Ravi and id - 123

No comments.

6. Maximum Array Correlation

Correct

Coding | Sorting | Greedy Algorithms | Medium | Two Pointers

Question description

For two arrays *a* and *b* of equal length, the array correlation is defined as the sum of all values *b[i]* where *b[i]* is greater than *a[i]*.

Given two integer arrays *a* and *b* of the same length, rearrange the elements of array *b* to maximize the array correlation. Return the maximum possible array correlation value.

Note: You are not allowed to rearrange the elements of the array *a*.

Example:
n = 5, *a* = [1, 2, 3, 4, 5], *b* = [3, 5, 4, 6, 2]

b can be rearranged to [2, 3, 4, 5, 6]. Now, for each index, the integers in *b* are greater than those in *a*. The array correlation is 2 + 3 + 4 + 5 + 6 = 20.

Function Description

Complete the function *getMaxArrayCorrelation* in the editor with the following parameter(s):

a[n]: the fixed order array

b[n]: the array to reorder

Returns:

long int: the maximum possible array correlation

Constraints

- $1 \le n \le 2 \times 10^5$
- $1 \le a[i], b[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, that represents the number of elements in *a*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *a[i].*
The next line contains an integer, *n*, that represents the number of elements in *b*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *b[i].*

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     ----------
5       →  a[] size n = 5
1       →  a = [1, 4, 2, 1, 3]
4
2
1
3
5       →  b[] size n = 5
2       →  b = [2, 3, 1, 2, 2]
3
1
2
2
```

Sample Output

```
7
```

Explanation

The second array can be rearranged to *b = [2, 1, 3, 2, 2]*. *b[i] > a[i]* for *i* = 0, 3, and 4, and the array correlation is 2 + 3 + 2 = 7.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     ----------
4       →  a[] size n = 4
1       →  a = [1, 9, 4, 2]
9
4
2
4       →  b[] size n = 4
8       →  b = [8, 4, 3, 1]
4
3
1
```

Sample Output

```
15
```

Explanation

The second array can be rearranged to *b = [3, 1, 8, 4]*. *b[i] > a[i]* for *i* = 0, 3, and, 4. The array correlation is 3 + 8 + 4 = 15.

Interviewer guidelines

▼ SOLUTION

Skills: Greedy Algorithms, Sorting

Optimal Solution:

We can use a greedy approach here. We will always try to take the maximum values of b as much as possible. We can sort the two arrays in reverse order and iterate through the array *a.* We add *b[j]* to the score and increment *j* at each element where *a[i] < b[j].*

Python Solution:

```
def getMaxArrayCorrelation(a, b):
    list.sort(a, reverse = True)
    list.sort(b, reverse = True)
    ans = 0
    i, j = 0, 0
    n = len(a)
    while i < n and j < n:
        if a[i] < b[j]:
            ans += b[j]
            j += 1
        i += 1
    return ans
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n logn)

Sorting the arrays takes O(n logn) time in the worst case.

Space Complexity - O(n)

Assuming that the inbuilt sorting functions use a variation of merge sort, sorting uses O(n) extra space.

Candidate's Solution

Language used: **Java 17**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMaxArrayCorrelation' function below.
```

```
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY a
     *  2. INTEGER_ARRAY b
     */

    public static long getMaxArrayCorrelation(List<Integer> a, List<Integer> b) {

        int n = a.size();

        List<Integer> sortedA = new ArrayList<>(a);
        List<Integer> sortedB = new ArrayList<>(b);

        Collections.sort(sortedA);
        Collections.sort(sortedB);

        int i = n-1;
        int j = n-1;

        long sum = 0;

        while(i >= 0 && j >= 0){
            if( sortedB.get(j) > sortedA.get(i)){
                sum += sortedB.get(j);
                i--;
                j--;
            } else {
                i--;
            }
        }


        return sum;



    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
```

```
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int aCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> a = IntStream.range(0, aCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int bCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> b = IntStream.range(0, bCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.getMaxArrayCorrelation(a, b);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

Testcase 0	Easy	Sample	Success	1	0.0518 sec	33.5 KB
Testcase 1	Easy	Sample	Success	1	0.0516 sec	33.7 KB
Testcase 2	Easy	Sample	Success	1	0.063 sec	33.6 KB
Testcase 3	Easy	Hidden	Success	3	0.0513 sec	33.5 KB
Testcase 4	Easy	Hidden	Success	3	0.056 sec	33.7 KB
Testcase 5	Medium	Hidden	Success	4	0.0565 sec	34.2 KB
Testcase 6	Medium	Hidden	Success	5	0.0773 sec	36 KB
Testcase 7	Medium	Hidden	Success	6	0.0929 sec	37.8 KB
Testcase 8	Medium	Hidden	Success	6	0.0973 sec	38 KB
Testcase 9	Hard	Hidden	Success	7	0.2157 sec	57.9 KB
Testcase 10	Hard	Hidden	Success	7	0.3442 sec	62.4 KB
Testcase 11	Hard	Hidden	Success	7	0.3168 sec	63.7 KB
Testcase 12	Hard	Hidden	Success	7	0.5129 sec	90.5 KB

Testcase 13	Hard	Hidden	Success	8	0.7588 sec	171 KB
Testcase 14	Hard	Hidden	Success	9	0.6032 sec	158 KB

No comments.

7. Java: Optimized Sequential Video Watching

Correct

Coding Medium Greedy Java

Question description

A learner is given a sequential playlist of *n* videos to enhance their knowledge. Implement a function to calculate the minimum time required to complete a total of *m* viewings while adhering to the platform's constraints.

System Constraints

- Sequential Viewing:
  - The learner must watch videos in the order they appear in the playlist.
  - To watch a video at index *i*, the learner must first watch all videos from index 0 to *i-1* at least once.
- Initial Full Viewing:
  - To view any video at index *i* for the first time, the learner must spend *firstWatch[i] + repeatWatch[i]* minutes. This ensures a complete understanding of the video.
- Subsequent Viewings:
  - For subsequent viewings of a video at index *i*, the learner spends only *repeatWatch[i]* minutes.
  - At any time, they can rewatch any video they have already watched.
- Minimum Total Watch Count:
  - The learner must achieve at least *m* total viewings across all videos.
  - Videos can be rewatched as desired, and not all videos need to be watched equally.

Return the minimum time required to complete *m* viewings of videos while following these constraints.

Example

If *n* = 4, *m* = 4, *firstWatch* = [1, 5, 9, 11], and *repeatWatch* = [2, 7, 10, 11], the optimal strategy is as follows:

- The learner watches the 0th video for the first time, spending *firstWatch*[0] + *repeatWatch*[0] = 1 + 2 = 3 minutes.
- The learner then rewatches the 0th video 3 more times in, spending *repeatWatch*[0]*3 = 2 * 3 = 6 minutes.
- The learner has successfully completed 4 viewings. Thus, the total time required is 3 + 6 = 9 minutes, and 9 is returned.

Function Description

Complete the function *findMinMinutes* in the editor below.

Function Parameters

int m: the total number of video viewings to complete

int firstWatch[n]: the minutes required to watch each video the first time

int repeatWatch[n]: the minutes required to rewatch each video

Returns

long: the minimum minutes required for the learner to reach their target number of viewings

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 10^9$
- $1 \le firstWatch[i], repeatWatch[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *m*.
The next line contains an integer, *n,* the size of *firstWatch*.
Each of the next *n* lines contains an integer, *firstWatch[i]*.
The next line contains an integer, *n,* the size of *repeatWatch*.
Each of the next *n* lines contains an integer, *repeatWatch[i]*.

▼ SAMPLE CASE 0

Sample Input 0

```
STDIN    FUNCTION
-----    --------
5        m = 5
3        firstWatch[] size n = 3
6        firstWatch = [6, 8, 9]
8
9
3        repeatWatch[] size n = 3
4        repeatWatch = [4, 2, 10]
2
10
```

Sample Output 0

```
26
```

Explanation

The goal is to complete 5 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time, so the time required is *firstWatch*[0] + *repeatWatch*[0] = 6 + 4 = 10 minutes.
- The learner watches the 1st video for the first time, so the time required is *firstWatch*[1] + *repeatWatch*[1] = 8 + 2 = 10 minutes.
- The learner then rewatches the 1st video 3 times. So the time taken = *repeatWatch*[1]*3 = 2 * 3 = 6 minutes.
- The total time required to complete 5 video viewings is 10 + 10 + 6 = 26 minutes.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
6         m = 6
5         firstWatch[] size n = 5
2         firstWatch = [2, 3, 7, 8, 4]
3
7
8
4
5         repeatWatch[] size n = 5
3         repeatWatch = [3, 6, 8, 7, 6]
6
```

```
8
7
6
```

Sample Output 1

```
20
```

Explanation

The goal is to complete 6 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time in *firstWatch*[0] + *repeatWatch[0]* = 2 + 3 = 5 minutes.
- The learner then rewatches the 0th video 5 times in *repeatWatch[0]**5 = 3 * 5 = 15 minutes.
- The total time required to complete 6 viewings is 5 + 15 = 20 minutes.

Interviewer guidelines

▼ SOLUTION

Description

The greedy solution suggests that, at any given index, if the time to watch the video is less than at any other index between i+1 and n, the minimum total time will be the sum of the watch time up to index i-1, plus the repeat time of the ith video multiplied by (m-i).

Solution

```
class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
        long minMinutes = Long.MAX_VALUE;
        long cur = 0;
        int n = firstWatch.size();

        for (int i = 0; i < Math.min(n,m); i++) {
```

```
            int a = firstWatch.get(i);
            int b = repeatWatch.get(i);

            minMinutes = Math.min(minMinutes, cur + ((long) (m - i) * b) + a);

            cur += a + b;
        }
        return minMinutes;
    }

}
```

▼ COMPLEXITY

Time Complexity - O(n)

The algorithm iterates through each number in *firstWatch* and *repeatWatch* exactly once, performing constant-time checks and arithmetic operations in each iteration. Thus, the time complexity is O(n), where *n* is the length of *firstWatch*.

Space Complexity - O(1)

Space usage is constant, only utilizing a fixed number of variables (*mn, cur,* etc.) that do not scale with the length of *firstWatch*. Therefore, the space complexity is O(1).

Candidate's Solution

Language used: **Java 17**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'findMinMinutes' function below.
```

```
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {

        int n = firstWatch.size();
        if( m <= 0) return 0;

        long[] unlock = new long[n+1];
        long[] minRepeat = new long[n+1];

        long curUnlock = 0;
        long curMinR = Long.MAX_VALUE;

        for(int k = 1; k <= n; k++){
            int i = k - 1;
            curUnlock += (long) firstWatch.get(i) + repeatWatch.get(i);
            unlock[k] = curUnlock;

            curMinR = Math.min(curMinR, (long) repeatWatch.get(i));
            minRepeat[k] = curMinR;

        }

        long answer = Long.MAX_VALUE;
        for(int k = 1; k <= n; k++){
            long extras = Math.max(0l, (long) m - k);
            long totalTime = unlock[k] + extras * minRepeat[k];

            if(totalTime < answer){
                answer = totalTime;
            }
        }
        return answer;
    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int m = Integer.parseInt(bufferedReader.readLine().trim());

        int firstWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> firstWatch = IntStream.range(0, firstWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int repeatWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> repeatWatch = IntStream.range(0, repeatWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.findMinMinutes(m, firstWatch, repeatWatch);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

```
102
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 2	Easy	Sample	Success	1	0.0774 sec	33.5 KB
Testcase 0	Easy	Sample	Success	1	0.0514 sec	33.7 KB
Testcase 1	Easy	Sample	Success	1	0.0561 sec	33.8 KB
Testcase 3	Easy	Hidden	Success	4	0.0479 sec	33.9 KB
Testcase 4	Easy	Hidden	Success	4	0.0657 sec	34.5 KB
Testcase 5	Easy	Hidden	Success	4	0.1199 sec	42.1 KB
Testcase 6	Medium	Hidden	Success	5	0.2702 sec	63.1 KB
Testcase 7	Medium	Hidden	Success	5	0.6842 sec	158 KB
Testcase 8	Medium	Hidden	Success	5	0.5771 sec	167 KB
Testcase 9	Medium	Hidden	Success	5	0.5776 sec	168 KB
Testcase 10	Medium	Hidden	Success	5	0.5498 sec	168 KB
Testcase 11	Medium	Hidden	Success	5	0.4258 sec	156 KB

Testcase 12	Hard	Hidden	Success	10	0.2498 sec	63.9 KB
Testcase 13	Hard	Hidden	Success	10	0.4636 sec	139 KB
Testcase 14	Hard	Hidden	Success	10	0.0604 sec	33.8 KB

No comments.